ScamShield Agent ■■

Google Cloud Rapid Agent Hackathon 2026 — Partner Track Guide

Conceptual Analogy: The Cyber-Detective Agency

To easily explain how this upgraded system works, think of the application as a **Cyber-Detective Agency**. Each part of our MERN stack and Agentic pipeline has an essential conceptual role:

- ■■■■■ **React Frontend (The Storefront & Reception Desk)**: The visual office where the customer stands, submits a suspicious letter, and views active results.
- ■■■■■ **Express Backend (The Office Manager)**: Stands behind the desk, receives requests, routes files, and communicates with the back room.
- ■■■■■ **MongoDB Atlas (The Filing Cabinets)**: The storage room where the history of threat patterns and signatures are locked in folders.
- ■■■■■ **Google ADK Agent / Gemini 2.5 Flash (The Lead Detective)**: Hired by the Office Manager. The Detective doesn't just read the letter and guess—they have a **toolkit (MCP Server)** containing keys to the **Filing Cabinets (MongoDB)**. The Detective actively searches past logs, evaluates risk, logs new records into the files, and compiles the final 6-step progress report for the manager.

1. System Architecture & Context

ScamShield Agent is upgraded from a single-step API chatbot to a production-grade **autonomous multi-step agent**. The architecture is designed to satisfy the rigorous technical requirements of the Google Cloud Rapid Agent Hackathon and the MongoDB Partner Track.

React Frontend (:3000)	Express Backend (:5000)	Python ADK Bridge (:8080)	MongoDB MCP Server
Sleek glassmorphism UI showing threat analysis panels	Routes the API requests, serves endpoints & FIR complaints	Vertex AI Google ADK agent with fastapi bridge	Enables direct DB search & persist tools inside AI reasoning

2. The 6-Step Multi-Step Agent Pipeline

Instead of returning a static response, the backend and agent coordinate an active multi-turn pipeline:

- **Step 1 — Entity Extraction**: The model extracts domains, company names, recruiter identities, payment amounts, and whatsapp contact phone numbers.
- **Step 2 — Threat Ledger MCP Search (MongoDB Tool)**: The Google ADK Agent invokes the MongoDB *aggregate* tool directly inside its reasoning loop to find match criteria against historical scam records.
- **Step 3 — Trust Index Calculation**: Evaluates urgency phrases, payment triggers, and domain matches to resolve a Trust Score between 0 and 100.
- **Step 4 — AI Threat Verdict**: Gemini 2.5 Flash calculates the final riskLevel (LOW, MEDIUM, HIGH, CRITICAL) and scam category.
- **Step 5 — Telemetry Logging (MongoDB Tool)**: The agent automatically triggers the *insert-many* MongoDB MCP tool to save the investigation telemetry record.
- **Step 6 — Complaints Escalation**: Automatically formats a complaint draft customized for Indian authorities (cybercrime.gov.in) pre-filled with all extracted scam details.

3. Core Code Files & Integrations

File 1: agent.py (The ADK Python Bridge)

This file defines the Google ADK Agent structure and connects it to the MongoDB MCP server tools. It exposes a uvicorn FastAPI server on port 8080. Here is how the MCP toolset is registered:

```
from google.adk.tools import McpToolset, StdioConnectionParams from mcp import
StdioServerParameters def build_mcp_toolset(): return McpToolset(
connection_params=StdioConnectionParams( server_params=StdioServerParameters(
command='npx', args=['-y', 'mongodb-mcp-server', f'--mongodb-uri={MONGODB_URI}'] ) ) )
```

File 2: server/services/agentService.js (Backend Connector)

Exposes a dual-layered client. It makes HTTP POST requests to port 8080 to invoke the ADK pipeline. To ensure robust production behavior, if the ADK server is waking up or unreachable, it gracefully falls back to direct Gemini API requests so the app never breaks:

```
async function analyzeViaBestAvailable(text) { const adkAlive = await isADKBridgeAlive();
if (adkAlive) { try { const result = await callADKAgent(text); return { result, usedADK:
true }; } catch (err) { console.warn('ADK failed, falling back to Gemini direct:',
err.message); } } const result = await analyzeWithGemini(text); return { result, usedADK:
false }; }
```

File 3: client/src/components/AgentStatusPanel.jsx (Status UI)

A floating glassmorphism UI element that renders a real-time terminal feed. It connects directly to the server's streaming chunks and displays status updates, allowing judges to visually track the ADK agent's tool calls and reasoning phases in real time.

4. Local Verification & Demo Strategy

To run your local test environment, execute these commands in three separate terminal screens:

```
# Terminal 1: Start ADK bridge $env:PYTHONUTF8=1; venv\Scripts\python agent.py serve #
Terminal 2: Start Node.js backend cd server && npm run dev # Terminal 3: Start Vite client
cd client && npm run dev
```

■ Expert Demo Video Strategy (3 Minutes):

- **0:00 - 0:30 (The Hook)**: State the critical problem (India's job seekers losing lakhs to onboarding fee scammers) and introduce ScamShield Agent.
- **0:30 - 1:30 (The Live Flow)**: Input a sample onboarding fee scam text. Point to the floating **Agent Status Panel** on the right side as it lights up, demonstrating the multi-step ADK reasoning and MongoDB MCP ledger tool calls.
- **1:30 - 2:30 (Extracted Entities & Legal Draft)**: Show the resolved Trust Score, categories, and click 'Download FIR Template' to showcase the auto-filled draft for cybercrime.gov.in.
- **2:30 - 3:00 (The Pitch)**: Summarize how the solution satisfies the Google Cloud ADK agent rules and integrates MongoDB MCP tools directly in the reasoning loop. Close with your mission: democratizing security for India's youth!